Webpack 原理解析 - 编译完成阶段

模块编译完成阶段，`webpack` 的主要做了以下几件事：

1. 根据模块编译阶段收集的 `entry` 和 `module` 内容，处理它们之间的依赖关系，组合成最终输出的 `chunk` 模块。

实现Chunk模块组装方法（3-5）

```
// core/compilation.js
const fs = require('fs')

class Compilation {
  constructor(compiler) {
    /**
     * ********************
     */
  }

  /**
   * 编译并保存入口文件对象
     */
  buildEntryModule(entry) {
    Object.keys(entry).forEach((entryName) => {
      const entryPath = entry[entryName]
      const entryObj = this.buildModule(entryName, entryPath)
      this.entries.add(entryObj)

      // 开始组装chunk
      this.buildChunk(entryName, entryObj)
    })
  }

  /**
   * ********************
     */

  /**
   * 根据入口文件和依赖模块组装chunks
   * @param {*} entryName 入口名称
   * @param {*} entryObj 入口对象
```

```
33      */
34    buildChunk(entryName, entryObj) {
35      const chunk = {
36        name: entryName, // 每个入口文件作为一个独立的chunk
37        entryModule: entryObj, // entry编译后的对象
38        modules: Array.from(this.modules).filter((module) =>
    module.name.includes(entryName)) // 寻找与当前entry有关的module
39      }
40
41      // 将组装好的chunk添加到chunks中
42      this.chunks.add(chunk)
43    }
44
45    /**
46     * 处理module到chunk的转化
47     * @param {*} callback 转化完成后的回调方法
48     */
49    seal(callback) {
50      this.buildEntryModule(this.compiler.entry)
51    }
52  }
53
54  module.exports = Compilation
```

- `buildChunk` 方法用于根据当前入口文件名称，过滤出this.modules中所有包含入口文件名称的依赖模块。

---

chunk模块最终输出内容

```
1   Set {
2     name: 'main',
3     entryModule: {
4       id: './src/index.js',
5       dependencies: Set(1) { './src/module/module1.js' },
6       name: [ 'main' ],
7       _source: 'const {\n' +
8         '  module1\n' +
9         '} = __webpack_require__("./src/module/module1.js");\n' +
10        "console.log('chaser => module1', module1);\n" +
11        "console.log('chaser => init');\n" +
12        'console.log(process.argv.slice(2), __dirname);\n' +
13        "const loader = 'loader join code';\n" +
14        "const loader2 = 'loader2 join code';"
15    },
```

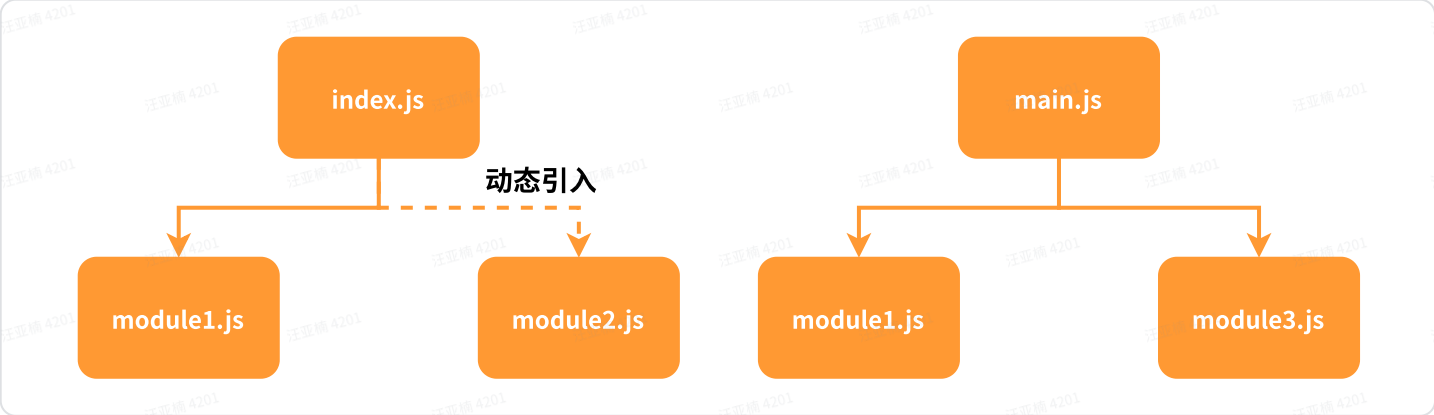
```
    modules: [
      {
        id: './src/module/module2.js',
        dependencies: Set(0) {},
        name: [Array],
        _source: "const module2 = '模块2';\n" +
          'exports.module2 = module2;\n' +
          "const loader = 'loader join code';\n" +
          "const loader2 = 'loader2 join code';"
      },
      {
        id: './src/module/module1.js',
        dependencies: [Set],
        name: [Array],
        _source: 'const {\n' +
          '  module2\n' +
          '} = __webpack_require__("./src/module/module2.js");\n' +
          'console.log(module2, ...arguments);\n' +
          "const module1 = '模块1';\n" +
          'exports.module1 = module1;\n' +
          "const loader = 'loader join code';\n" +
          "const loader2 = 'loader2 join code';"
      }
    ]
  }
```

这一步的关键逻辑是将 `module` 按规则组织成 `chunks` ，webpack 内置的 `chunk` 封装规则比较简单：

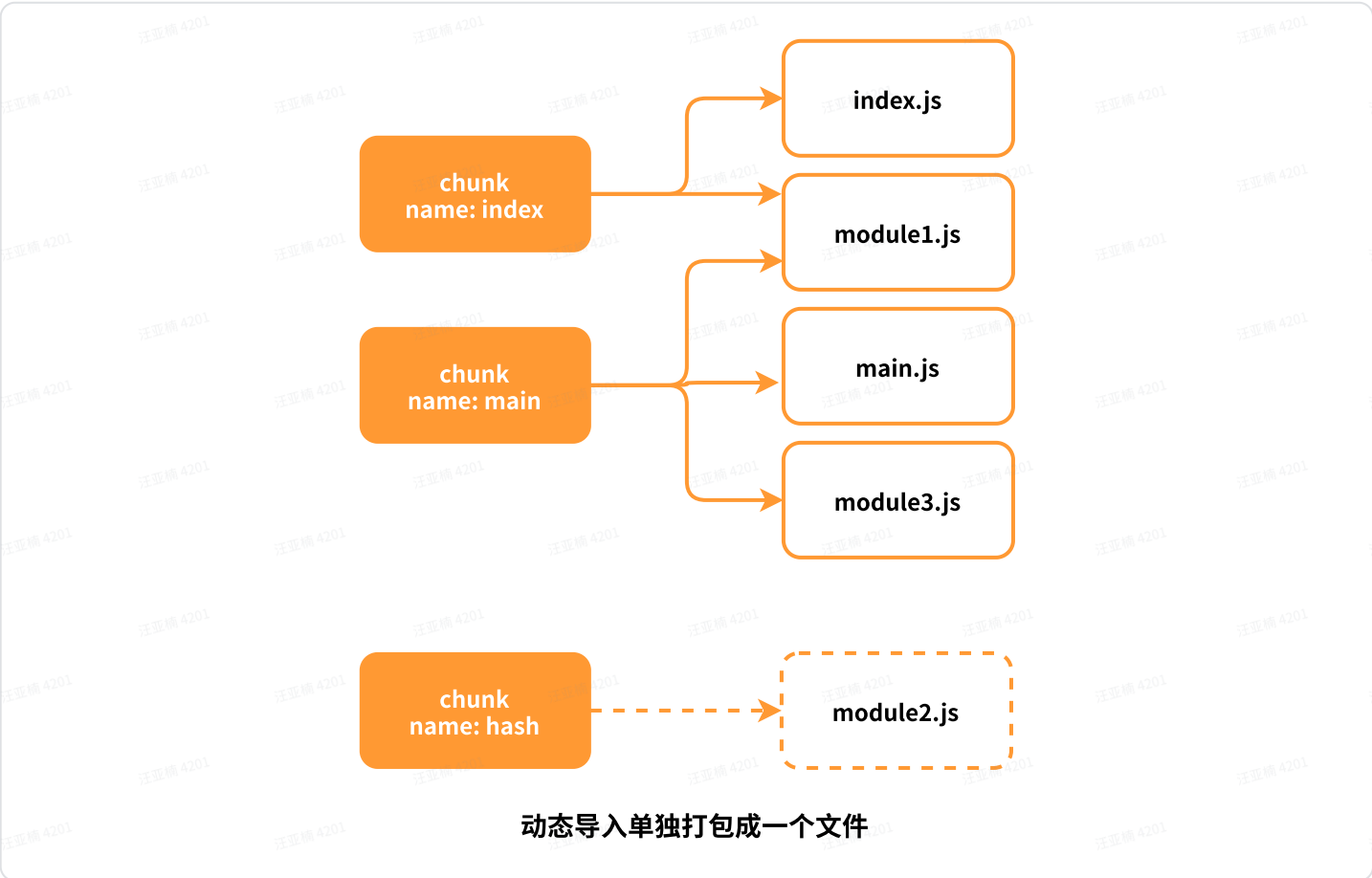
- `entry` 及 entry 触达到的模块，组合成一个 `chunk`。
- 使用动态引入语句引入的模块，各自组合成一个 `chunk`。

`chunk` 是输出的基本单位，默认情况下这些 `chunks` 与最终输出的资源一一对应，按上面的规则大致上可以推导出一个 `entry` 会对应打包出一个资源，而通过动态引入语句引入的模块，也对应会打包出相应的资源。

实例流程图：



根据上面说的规则，最终chunks会生成一下结构：



END